Asking someone to pay is common. It's never had a clean digital path.

A daughter at Croma. A spouse at a restaurant. An accountant forwarding an invoice. It happens constantly — offline and informally.

The need is persistent

- **Priya** on Croma checkout — UPI has failed twice; asks her father.
- **Meera** at The Bombay Canteen — scans the bill QR, asks her husband.
- **Anita** (accountant) forwards a Razorpay Payment Link to her CFO.
- Today this resolves as a phone call followed by a separate UPI transfer — **off Razorpay's rails**, no order linkage, no audit trail.

No existing primitive fits

- **UPI P2P collect** came closest — **banned by NPCI Oct 2025** after widespread fraud abuse.
- **UPI Circle** addresses a narrow slice — pre-registered trusted users with spending limits. Requires setup, no merchant context, not designed for ad-hoc delegation at the payment moment.
- Everything else: phone call + manual transfer. No merchant context, no trust signals, no audit trail, no receipt.

> **Why Razorpay:** this fits Razorpay's existing surfaces (Checkout, POS QR, Payment Link), brings off-rail volume on-rail, and — because delegated payments emit the same webhooks as direct ones — reaches merchants' backends without integration work.

→ **The open question is adoption, not whether the need exists.** Only real-world usage can answer it — engineering alone can't.

One delegation primitive. Three entries. One approval page.

Working end-to-end prototype — React frontend, Express/Node backend, SQLite. Walkable browser demo covering all 5 scenarios.

Entry surfaces
A: Checkout SDK · B: QR Landing · C: Payment Link

`| POST /v1/delegations ▼`

Delegation primitive
Bound to an order_id · Advisory tier T1–T4 assigned server-side · One-open-per-order (DB-enforced) · State machine + TTL

`| Share via WhatsApp / SMS (requestor's own device) ▼`

Approver page — rzp.io/r/{id}
Razorpay-hosted · tier-adaptive trust panel · approver pays with their own method

`| Standard Razorpay rail ▼`

Webhooks to merchant
`payment.captured` · `order.paid` — identical to direct payments

No merchant code changes required. The last two webhook events are identical to direct payments. Merchant's existing handler fires unchanged — that's the adoption property.

Server-owned integrity. Advisory tier computed from Razorpay merchant data (never client-supplied). Preamble sanitized (URLs, phone numbers, RTL overrides, homoglyphs stripped, 140-char cap). System block tamper-proof.

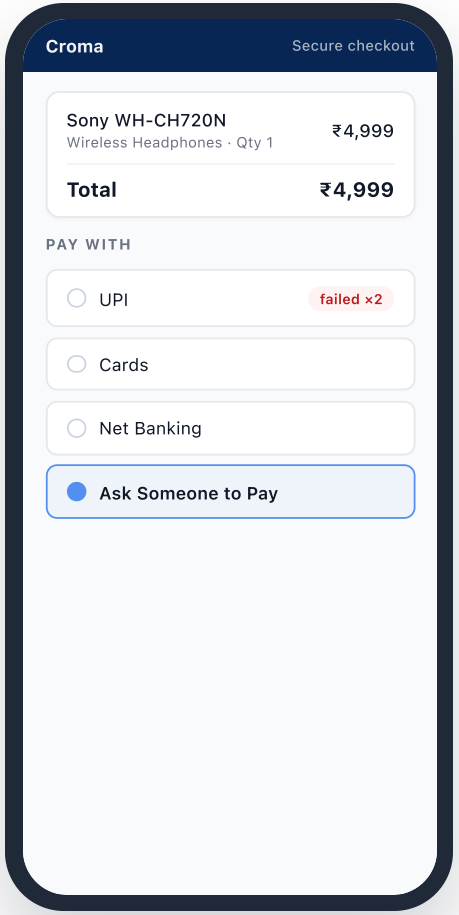
Success metrics (what real usage would need to show): adoption rate, approval rate, payment completion rate, time to resolution. Thresholds to be set against a live baseline.

What was built in this repo: React + Express/Node + SQLite. 5 demo scenarios across all entry points. Full delegation state machine, server-side tier assignment, anti-spoofing sanitization, webhook event log, "Under the Hood" developer panel.

→ **One primitive, three entry points, one approval page, one webhook contract.** Entry surfaces are decoupled from the primitive; the primitive is decoupled from the merchant's integration.

See the choice

On the merchant's own checkout surface — alongside the usual pay buttons.



UNDER THE HOOD

STATE — → —

API

```
POST /v1/orders (merchant, existing)
```

No delegation exists yet — the order is what "Ask Someone" will attach to.

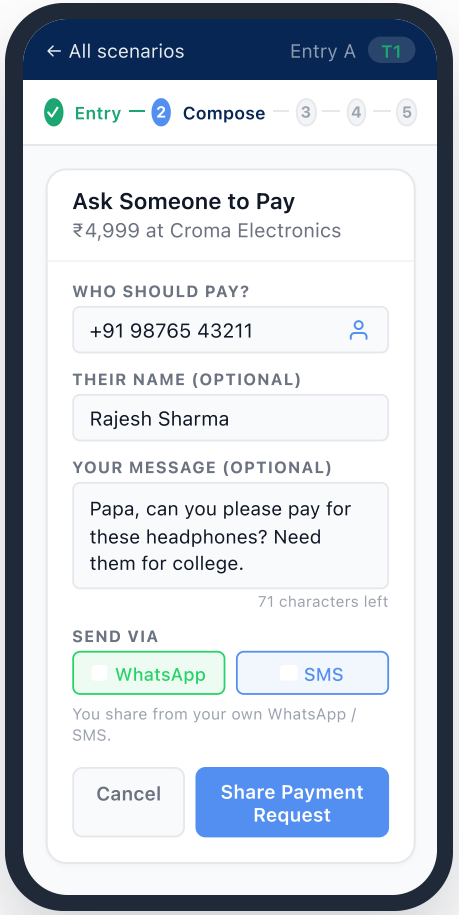
WEBHOOK None yet

SERVER **Eligibility gated upstream.** *Merchant age ≥ 30 days AND not risk-flagged. The button only renders for eligible merchants — ineligibility never reaches the delegation primitive.*

→ **Entry decides who's in.** If the merchant shouldn't be delegable, the button never appears. Delegation itself doesn't have to police who gets to use it.

Compose the delegation

Requestor picks the approver, writes a short note, shares.



UNDER THE HOOD

STATE

– → `created`

API

```
POST /v1/delegations
```

`{ order_id, approver_phone, entry_point, preamble_text, channel_whatsapp, channel_sms }`

WEBHOOK

`order.delegation_requested`

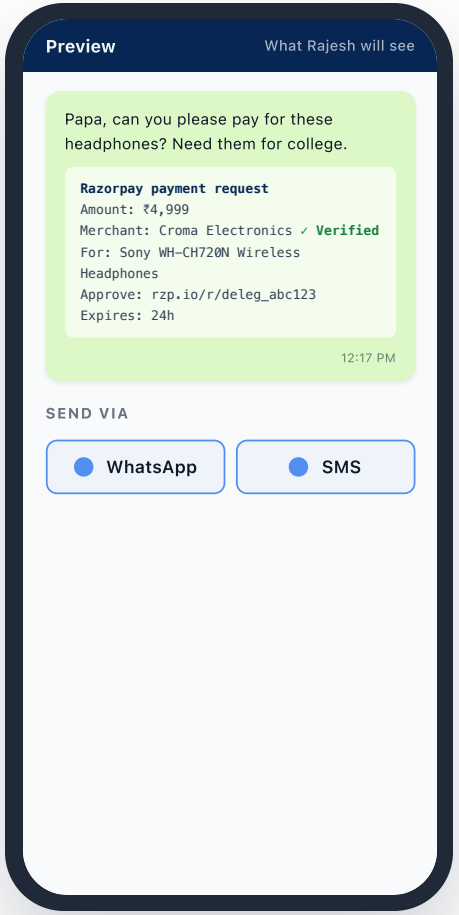
SERVER

One-open-per-order *enforced at the DB layer — returns* `409` *if violated.* **Preamble sanitized***: URLs, phone numbers, RTL overrides, homoglyphs stripped; 140-char hard cap.* **Advisory tier** *(value-add) assigned server-side from existing merchant data — never user-supplied.*

→ **Trust rides the delivery channel.** The approver recognizes the request because it arrives from a saved WhatsApp / SMS contact — their son asking, not a stranger with a link.

Preview what the approver receives

Before sharing, the requestor sees exactly what the approver will see.



UNDER THE HOOD

STATE

`created` → `shared`

API

```
POST /v1/delegations/:id/share
```

`{ channels: ["whatsapp", "sms"] }`

WEBHOOK

None new (created already fired `order.delegation_requested`)

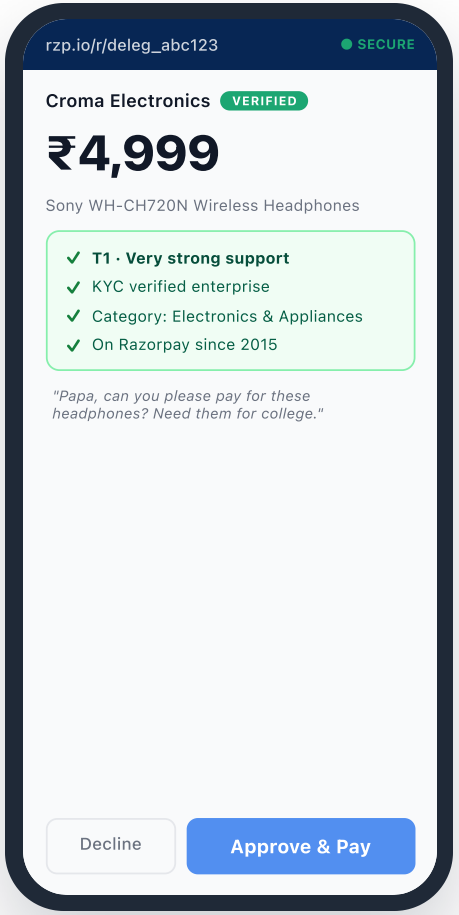
SERVER

System block is server-rendered and tamper-proof. *Preamble lives above, separated by two newlines.* **OG metadata** *for WhatsApp rich preview is served from the same fields — preview, delivered message, and approval page all render from one source.*

→ **One source of truth across preview, message, and approval page.** The requestor can't show Rajesh something different from what the approval page will say. That invariant is what makes the preamble safe to ship.

Approver opens the link

Razorpay-hosted page at `rzp.io/r/{id}`, tier-adaptive trust panel.



UNDER THE HOOD

STATE

`shared` → `opened`

API

```
GET /v1/delegations/:id
```

Auto-expires if TTL passed. Re-renders system block server-side on every open.

WEBHOOK

None — opens aren't merchant-visible

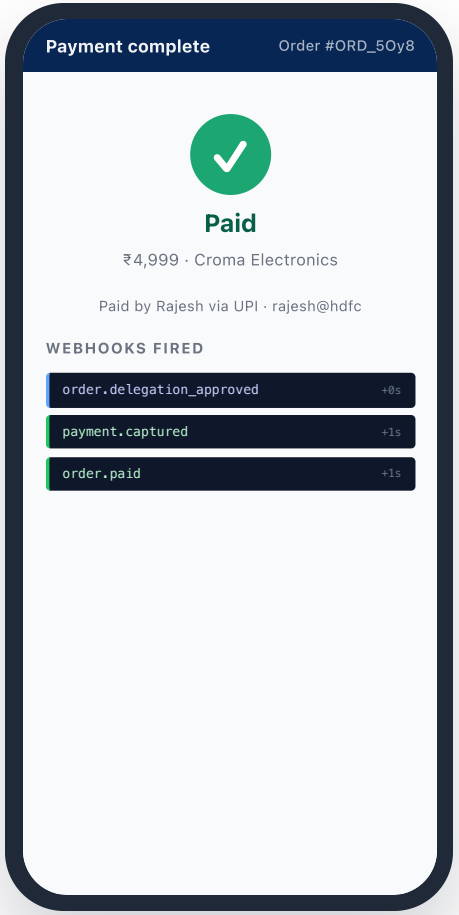
SERVER

Advisory tier assigned server-side *from merchant signals (KYC, MCC, tenure). T1 shows line items + verified badge; T4 shows caution banner. Tier never comes from the client — the frontend renders what the server decides.*

→ **Trust panel is additive, not load-bearing.** Razorpay already controls who reaches this page — the panel adds context for the approver, it doesn't gate the transaction.

Paid — merchant gets standard webhooks

Identical events to direct payments. Merchant's existing handler fires unchanged.



UNDER THE HOOD

STATE

`opened` → `captured`

API

```
POST /v1/delegations/:id/pay → /capture
```

Standard Razorpay payment rails from here. Delegation is closed; payment records attach `delegation_id`.

WEBHOOK

```
order.delegation_approved
payment.captured
order.paid
```

SERVER

The last two events are **identical to direct-payment webhooks**. *Merchant's existing handler processes this order like any other.* `delegation_id` *rides in* `notes` *for reconciliation if the merchant wants it.*

→ **No merchant code changes.** That's the adoption story — this primitive reaches every Razorpay merchant's backend without integration work.

What's in this repository

Runtime: React 18 + Vite frontend, Node.js + Express backend, SQLite (`better-sqlite3`). One process, no external dependencies.

Backend (src/backend/)

- **routes/delegation.js** — 9 endpoints covering the delegation lifecycle.
- **routes/scenarios.js** — 5 demo scenarios + per-scenario reset for replay.
- **routes/webhooks.js** — read-only inspection of events fired during the demo.
- **services/delegationMessageService.js** — preamble sanitization + server-generated system block.
- **db.js** — schema for merchants, orders, payment_links, qr_sessions, delegations, webhook_events. Unique index `one_open_delegation_per_order`.
- **assignTrustTier()** — T1–T4 tier computed server-side from KYC, MCC, tenure, risk flags. **Never client-supplied.**

Frontend (src/)

- **App.jsx** — toggles between ScenarioLauncher and DemoFlow.
- **DemoFlow.jsx** — orchestrates the 5 steps for a chosen scenario.
- **EntryCheckout · EntryQRLanding · EntryPaymentLink** — the three entry surfaces.
- **DelegationCompose · WhatsAppPreview · ApproverPage · StatusPage** — the delegation UI.
- **TrustPanel.jsx** — advisory panel, tier-adaptive.
- **HandshakePanel.jsx** — the "Under the Hood" developer panel visible on every step.
- **WebhookLog.jsx** — fixed bottom panel showing webhook events as they fire.

→ **One primitive, not three parallel features.** The delegation table and state machine are shared across all entry points. Entry point is a field, not a code path.

Delegations table

One row per delegation. Bound to an existing `order_id`. State machine fields alongside audit fields.

COLUMN	TYPE	NOTE
`id` PK	varchar(32)	*deleg_abc123xyz*
`order_id` FK	varchar(32)	*Foreign key to orders*
`requestor_id`	varchar(64)	*Merchant-provided customer id*
`approver_contact`	varchar(20)	*+91 phone number*
`approver_name`	varchar(64) · nullable	*Optional salutation label*
`preamble`	text · nullable	*Sanitized, 140 char cap*
`status`	enum	*created · shared · opened · paid · captured · declined · expired*
`advisory_tier`	enum	*T1 · T2 · T3 · T4 — server-computed*
`entry_point`	varchar(32)	*checkout_sdk · pos_qr · payment_link*
`payment_id`	varchar(32) · nullable	*Set on capture*
`created_at · expires_at`	bigint	*Unix epoch*
`ttl_seconds`	int	*86400 web · 900 POS · inherits for Payment Link*

Unique index: `one_open_delegation_per_order`. A second open delegation on the same order returns `409 ORDER_HAS_OPEN_DELEGATION`.

Tier is never user-supplied. Derived from merchant KYC, MCC, tenure, and risk flags in `assignTrustTier()`. The client cannot coerce it.

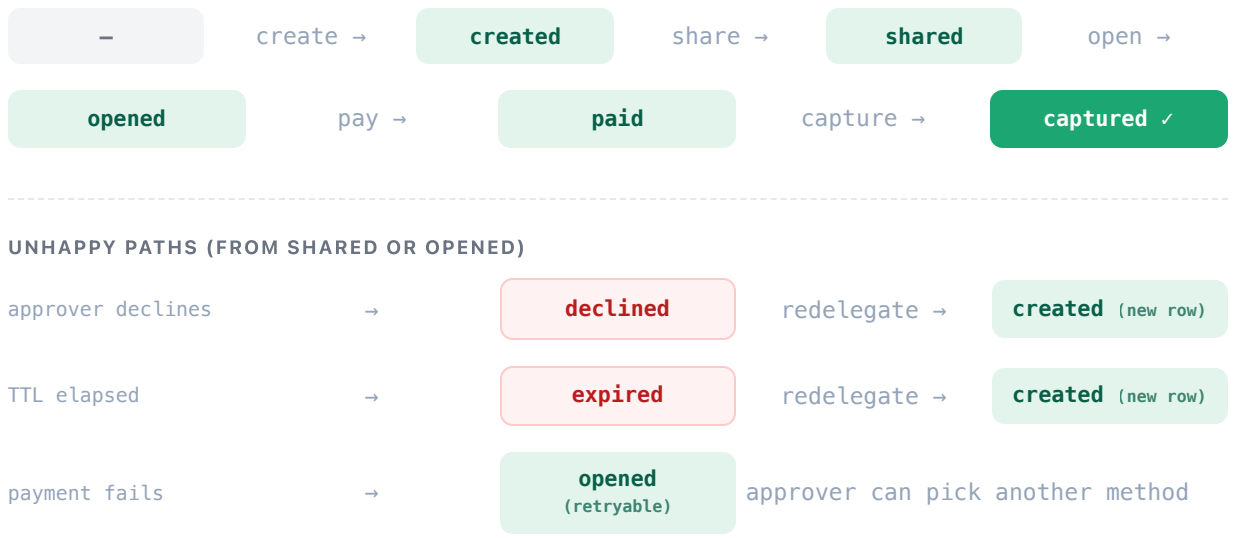
TTL is a request-validity window, not a payment SLA. It governs how long the approver has to act — 24 h for web, 15 min for POS, inherits the Payment Link's own expiry.

What the merchant sees: nothing from this table directly. They get the webhook events; `delegation_id` rides in `notes` for reconciliation.

→ **One open delegation per order — enforced at the DB layer.** Application code can't forget; the constraint is the invariant.

Delegation state machine

Happy path across the top; declined and expired drop back to `created` so the requestor can redelegate.



Transitions and webhooks

· → created `order.delegation_requested`
created → shared (no webhook)
shared → opened (no webhook)
opened → paid → captured `order.delegation_approved` · `payment.captured` · `order.paid`
→ declined `order.delegation_declined`
→ expired `order.delegation_expired`

Redelegate spawns a new delegation row for a different approver. The original row is terminal; history is preserved.

Payment failure is not a delegation state. A failed UPI auth keeps the delegation at `opened` — the approver retries with another method.

→ **Decline and expiry revert the order to `created`.** The requestor can try again — the delegation primitive is decoupled from the merchant's order lifecycle.

Delegation API — nine endpoints

New namespace `/v1/delegations`. One per state transition; `redelegate` handles the fork case.

METHOD	PATH	PURPOSE	STATE TRANSITION
POST	`/v1/delegations`	Create a delegation bound to an order_id. Server assigns tier. Enforces one-open-per-order.	`— → created`
GET	`/v1/delegations/:id`	Fetch with full context. Auto-expires if TTL passed. Re-renders system block server-side.	*read*
POST	`/v1/delegations/:id/share`	Mark shared. Records the channels used (whatsapp / sms).	`created → shared`
POST	`/v1/delegations/:id/open`	Approver opened the approval link.	`shared → opened`
POST	`/v1/delegations/:id/pay`	Approver selected a payment method. Hands off to standard Razorpay payment rail.	`opened → paid`
POST	`/v1/delegations/:id/capture`	Payment succeeded. Closes the delegation.	`paid → captured`
POST	`/v1/delegations/:id/fail`	Payment attempt failed (retryable). State stays at `opened`.	*retryable*
POST	`/v1/delegations/:id/decline`	Approver declined. Order reverts to `created`.	`* → declined`
POST	`/v1/delegations/:id/redelegate`	Closed delegation spawns a new one for a different approver.	`declined|expired → new created`

→ **Only `POST /v1/delegations` creates rows.** Every other endpoint operates on an existing delegation — that's the invariant that makes rate-limiting and audit clean.

Webhook contract

Four new delegation events. The payment result fires the same two events as direct payments — which is why merchants need no integration work.

NEW DELEGATION LIFECYCLE EVENTS

`order.delegation_requested`

Fires on delegation create. Payload carries `delegation.id`, `status: PENDING`, `order.id`.

Lets the merchant reserve inventory or mark the order as awaiting a third-party approval.

`order.delegation_approved`

Fires when the approver submits payment and Razorpay authorizes it. Precedes `payment.captured`.

Confirms the delegation resolved positively. Optional for merchants.

`order.delegation_declined`

Fires when the approver declines. Order reverts to `created`.

Merchant can release held inventory or notify the buyer.

`order.delegation_expired`

Fires when TTL elapses without an approver decision. Order reverts to `created`.

Symmetrical to `declined` from the merchant's point of view.

STANDARD PAYMENT EVENTS — IDENTICAL TO DIRECT PAYMENTS

`payment.captured`

Standard Razorpay event. Payload is identical to direct-payment capture except `notes` carries `delegation_id` for optional reconciliation.

Merchant's existing capture handler fires unchanged.

`order.paid`

Standard Razorpay event. Order transitions to `paid` exactly as in a direct payment.

This is what fulfillment keys off — no handler change needed.

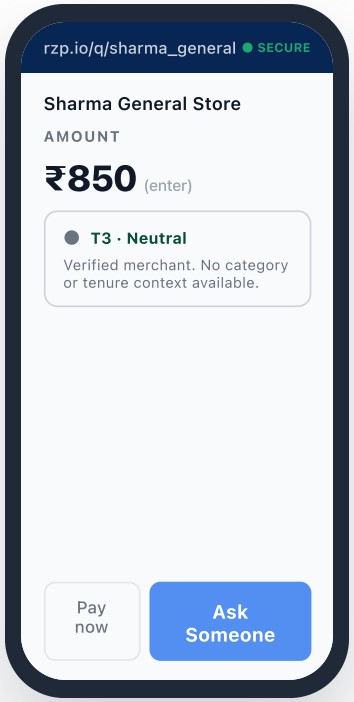
→ **The last two events are the adoption property.** A merchant who never registers the new delegation events still receives the payment as a normal capture. Delegation awareness is additive, not required.

In-store — dynamic and static QR

Same primitive, different entry. The QR encodes a Razorpay URL — not a `upi://` deep-link — so the landing page can offer "Ask Someone to Pay" alongside "Pay now".

SCENARIO 2 · DYNAMIC QR · T2

SCENARIO 3 · STATIC QR · T3



QR encodes a Razorpay URL, not `upi://`. Tradeoff: loses one-tap UPI app launch; gains a Razorpay-owned landing surface that can offer "Ask Someone to Pay" alongside "Pay now".

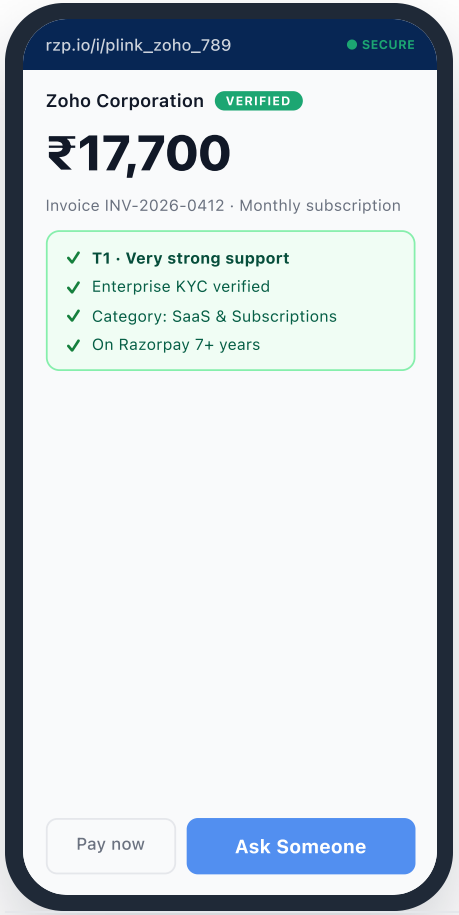
No POS firmware change. v1 scope explicitly excludes terminal UI updates. Cashier workflow unchanged; the delegation flow lives entirely on the customer's phone after scanning.

TTL: 15 min for in-store dynamic QR sessions (matches terminal countdown). Static QRs are perennial; TTL applies once a delegation is created.

→ **Depends on re-provisioning Razorpay's dynamic and static QR payloads to `rzp.io/q/...`** — platform-side change, not a firmware one. Flagged as a BLOCKER for Entry B.

B2B — accountant forwards to CFO

Payment Links are already URL-shareable. Adding "Ask Someone to Pay" on the payment page is a clean extension — the same primitive covers family and enterprise use cases.



Anita (accountant) taps "Ask Someone to Pay", enters Suresh's (CFO) phone, shares via WhatsApp. Suresh opens, sees Zoho's T1 trust panel, pays by netbanking. Zoho receives `payment.captured` + `order.paid`.

Same primitive. Different use case. The delegation table doesn't care whether the requestor is a teenager asking a parent or an accountant forwarding to a CFO.

TTL: inherits the Payment Link's own `expire_by`. B2B invoices often have longer windows than 24h — the delegation respects the merchant's expiry.

This is why the primitive is general. Scoping to "family payments" would have missed the invoice-approval use case that Payment Links already see today. One primitive covers both.

→ **Payment Links are the cleanest integration** — they're already URL-shareable. The "Ask Someone" button is additive on the existing payment page.

ADVISORY GRADIENT · SUPPORTING FEATURE

Tier is additive context, not a gate

Tier is computed server-side from merchant signals Razorpay already has. It shapes the approver page — it doesn't decide who gets to initiate a delegation.

T1 — VERY STRONG

KYC approved · line_items on order

- ✓ Merchant name + ✓ Verified badge
- ✓ Full line-item summary
- ✓ MCC category label
- ✓ Tenure on Razorpay

T2 — STRONG

KYC approved · MCC known · tenure ≥ 365d

- ✓ Merchant name + ✓ Verified badge
- — Line items — not in order
- ✓ MCC category label
- ✓ Tenure on Razorpay

T3 — NEUTRAL

Verified merchant · fallback tier

- ✓ Merchant name + ✓ Verified badge
- — Line items
- — Category
- — Tenure (not surfaced)

T4 — CAUTION

Risk-flagged OR tenure < 30d OR unverified

- ✓ ⚠ Warning banner on approval page
- ✓ Merchant name (no adjectives)
- ✓ Reason code from a fixed list
- ✓ Approver can still pay — it's advisory

Eligibility to initiate is gated upstream. Merchants with tenure < 30 days or high risk flag don't see the "Ask Someone" button at all. T4 exists for in-flight signal changes, not for gating entry.

Server-assigned, never client-supplied. The frontend renders what `assignTrustTier()` returns. A malicious caller cannot coerce a higher tier.

→ **The tier is a graduated honesty signal** — Razorpay shows less confidence when it has less data, rather than hiding the fact. The primary trust vector is still interpersonal: the approver knows the requestor.

Fraud & abuse mitigations

The primary risk vector is a malicious requestor using the preamble to impersonate Razorpay or misdirect the approver. Controls are layered at sanitization, rendering, and platform levels.

Anti-spoofing on the outgoing message

- **Server-side sanitization** — URLs, phone-number-shaped strings, Unicode bidi overrides (U+202A–E, U+2066–9), and homoglyphs of "Razorpay" all stripped before concatenation.
- **Hard separator** — two newlines between preamble and system block. System block always starts with the literal header.
- **URL locking** — exactly one URL allowed in the outgoing message: the canonical `rzp.io/r/{id}`. Anything else is removed.
- **Length cap** — preamble hard-capped at 140 chars; requests over return HTTP 400. Full message engineered to fit 2 GSM-7 segments.
- **DLT template gating** — SMS sends go through pre-registered DLT templates; tampering is rejected at carrier ingress.

Primitive-level integrity

- **One open delegation per order** — DB-level unique constraint. Second attempt returns `409 ORDER_HAS_OPEN_DELEGATION`.
- **Server-owned tier assignment** — tier comes from Razorpay's merchant data pipeline; client cannot coerce it.
- **Eligibility gated upstream** — merchants with tenure < 30 days or high risk flag can't initiate at all; the button doesn't render.
- **System block regenerated on every open** — no trust via link inspection; the approval page re-derives from fresh merchant data.
- **No requestor-name line on the approval page** — identity rides the delivery channel; Razorpay does not fabricate one.
- **Phased by trust tier** — proposed v1 rollout is to Razorpay-selected merchants only; biometric step-up and CAPTCHA are deferred scale-up levers, not v1 requirements.

→ **None of these controls rely on user vigilance.** They're structural — the preamble literally cannot reproduce the system block format, and a second delegation on the same order literally cannot be created.

What happens when things don't go clean

Declines, expiries, mid-flow payment failures — each has a state machine response and a webhook. The redelegate path keeps the order recoverable.

CASE	WHAT HAPPENS	STATE TRANSITION	WEBHOOK
Approver declines	Approver taps Decline (optional reason captured). Order reverts to `created`; requestor can redelegate to someone else.	`* → declined`	`order.delegation_declined`
Delegation expires	TTL elapses with no approver action. Landing page shows "This request has expired." Requestor can redelegate with a new TTL.	`* → expired`	`order.delegation_expired`
Payment fails mid-flow	Approver closes the UPI app without confirming, or the bank rejects. Delegation state stays at `opened`; approver can retry with a different method.	`opened` (unchanged)	*none from delegation side (standard `payment.failed` fires)*
Both channels tapped	Approver opens WhatsApp link, starts paying, then taps the SMS link. Second tap sees `status=processing` and no-ops.	`opened` (unchanged)	*none*
Requestor cancels	Requestor decides to pay directly or asks someone else. Self-decline via the same endpoint. Order reverts to `created`.	`* → declined`	`order.delegation_declined`
Redelegate	From a `declined` or `expired` delegation, spawn a new one for a different approver. Original row is terminal; history preserved.	`→ new created`	`order.delegation_requested`
Duplicate attempt	Creating a second open delegation on an order that already has one. DB unique constraint rejects.	*no state change*	`HTTP 409`

→ **Declined and expired are recoverable** via redelegate — the order stays alive. Payment failures don't burn the delegation; the approver just picks another method.

Key dependencies for production

The prototype is complete. Taking this to production would depend on the items below — grouped by owning team. Full matrix in `docs/DEPENDENCIES_AND_CLARIFICATIONS.md`.

Platform & infra

- **Webhooks team** — register the 4 new `order.delegation_*` events and commit payload schema.
- **API gateway** — route `/v1/delegations` namespace; rate-limit bucket; auth middleware.
- **Short URL** — allocate `rzp.io/r/*` and `rzp.io/q/*` namespaces.
- **Dynamic/static QR re-provisioning** — migrate payload from `upi://` to `rzp.io/q/...` URLs.
- **Checkout SDK** — feature-flagged "Ask Someone to Pay" button; compose modal (Razorpay-hosted vs embedded — pending decision).

Risk & data

- **Fraud/risk score API** — exact field name + endpoint for T4 warning overlay.
- **Merchant age exposure** — propagate `merchant.created_at` to all three entry gates.
- **Merchant trust data pipeline** — KYC, MCC, tenure surfaced to approver page.

Messaging & comms

- **DLT template approval** — 4 SMS templates (T1–T4), 7–14d carrier approval.
- **WhatsApp Business API templates** — 4 WABA templates, 1–3d approval.
- **WABA ownership** — single platform-owned WABA (proposed).
- **SMS sender ID & brand name** — DLT-approved.

Legal & regulatory (parallel track)

- **NPCI opinion** — formal confirmation that merchant-anchored delegation is distinct from the banned P2P collect. Does not gate engineering; gates production launch.
- **Chargeback T&C** — liability allocation between approver and merchant.
- **DPDP / DPO sign-off** — data classification and retention.
- **Approval page wording** — RBI consumer-protection review.

Open empirical questions

- **Adoption** — will users choose this over phone-call-plus-transfer?
- **Approval rate** — what % of shared delegations get approved?
- **Completion rate** — what % of approvals result in captured payments?
- **Time to resolution** — share to paid, end-to-end.